ESP32 SMART VISITOR COUNTER & ENERGY MANAGEMENT SYSTEM

Complete Arduino C++ Firmware & Technical Guide

Prepared for: **Harsh** | Target: ESP32 Dev Module | PCB: 120.65 mm × 66.04 mm

Project functions

Bi-directional people counting using two active-LOW IR beam sensors; OLED occupancy display; occupancy persistence in ESP32 NVS flash; status LED indication; and relay-based load control.

Important source note

The uploaded 3-page firmware document contains the architecture and most of the code, but its final OLED display function is cut off and the sensor routine uses short blocking delays. This PDF supplies a completed, non-blocking implementation while retaining the uploaded GPIO mapping and project behavior.

DESIGNED BY HARSH

1. System Overview

The ESP32 reads two IR sensors arranged in sequence across a doorway. If Sensor A is interrupted first and Sensor B follows, the firmware registers an entry and increments the count. If Sensor B is interrupted first and Sensor A follows, it registers an exit and decrements the count, without allowing the count to go below zero. The relay is ON while the count is greater than zero and OFF when the room is empty.

2. GPIO Mapping

Device / Signal	ESP32 GPIO	Mode / Setting
IR Sensor A (entry)	GPIO 14	Digital input, INPUT_PULLUP, active LOW
IR Sensor B (exit)	GPIO 27	Digital input, INPUT_PULLUP, active LOW
5V relay module input	GPIO 26	Digital output, active HIGH per source document
OLED SDA	GPIO 21	I²C data, address 0x3C
OLED SCL	GPIO 22	I²C clock
Status LED	GPIO 33	Digital output; use 330 Ω series resistor

3. Required Arduino Libraries

Install the ESP32 board package in Arduino IDE. Install these libraries through Library Manager: **Adafruit GFX Library** and **Adafruit SSD1306**. The Arduino-ESP32 core provides **Preferences.h** and **Wire.h**.

4. Direction Detection Logic

IDLE waits for the first beam. **A_TRIGGERED** waits for Sensor B and confirms entry. **B_TRIGGERED** waits for Sensor A and confirms exit. A 1500 ms sequence timeout cancels incomplete walk-bys. Sensor inputs are sampled and stable-state filtered without delay().

5. Complete Arduino C++ Source Code

Create a new Arduino sketch and paste the complete code below. Select the correct ESP32 Dev Module and COM port before uploading.

```
/*
  ESP32 Smart Visitor Counter & Energy Management System
  Designer: Harsh
  Target: ESP32 Dev Module

  GPIO map:
    IR Sensor A  -> GPIO 14 (active LOW)
    IR Sensor B  -> GPIO 27 (active LOW)
    Relay IN     -> GPIO 26 (active HIGH, as specified)
    OLED SDA     -> GPIO 21
    OLED SCL     -> GPIO 22
    Status LED   -> GPIO 33 (330 ohm series resistor)

  Libraries:
    Adafruit GFX
    Adafruit SSD1306
    Preferences (included with ESP32 Arduino core)
*/

#include <Arduino.h>
#include <Wire.h>
#include <Adafruit_GFX.h>
#include <Adafruit_SSD1306.h>
#include <Preferences.h>

// -------------------- Pin assignments --------------------
constexpr uint8_t SENSOR_A_PIN = 14;
constexpr uint8_t SENSOR_B_PIN = 27;
constexpr uint8_t RELAY_PIN = 26;
constexpr uint8_t STATUS_LED_PIN = 33;

// -------------------- OLED configuration ----------------
constexpr uint8_t SCREEN_WIDTH = 128;
constexpr uint8_t SCREEN_HEIGHT = 64;
constexpr int8_t OLED_RESET = -1;
constexpr uint8_t SCREEN_ADDRESS = 0x3C;

// -------------------- Timing parameters -----------------
constexpr uint32_t DEBOUNCE_MS = 35;
constexpr uint32_t SEQUENCE_TIMEOUT_MS = 1500;
constexpr uint32_t LED_PULSE_MS = 120;

Adafruit_SSD1306 display(
  SCREEN_WIDTH, SCREEN_HEIGHT, &Wire, OLED_RESET
);
Preferences preferences;

// -------------------- Occupancy / FSM --------------------
int occupantCount = 0;

enum DirectionState {
  IDLE,
  A_TRIGGERED,
  B_TRIGGERED
};

DirectionState currentState = IDLE;
uint32_t stateStartTime = 0;

// -------------------- Debounce state ---------------------
bool rawA = HIGH;
bool rawB = HIGH;
```

```
bool stableA = HIGH;
bool stableB = HIGH;

uint32_t rawAChangedAt = 0;
uint32_t rawBChangedAt = 0;

uint32_t ledTurnOffAt = 0;
bool ledPulseActive = false;

// -------------------- Function declarations --------------
void updateOLEDDisplay();
void saveOccupancyToFlash();
void handleDirectionalFSM();
void updateDebouncedInputs();
void updateStatusLED();
void updateRelay();

// -------------------- Setup ------------------------------
void setup() {
  Serial.begin(115200);

  pinMode(SENSOR_A_PIN, INPUT_PULLUP);
  pinMode(SENSOR_B_PIN, INPUT_PULLUP);
  pinMode(RELAY_PIN, OUTPUT);
  pinMode(STATUS_LED_PIN, OUTPUT);

  // Safe initial outputs (relay LOW = OFF for the specified module)
  digitalWrite(RELAY_PIN, LOW);
  digitalWrite(STATUS_LED_PIN, LOW);

  Wire.begin(21, 22);

  if (!display.begin(SSD1306_SWITCHCAPVCC, SCREEN_ADDRESS)) {
    Serial.println("ERROR: OLED initialization failed.");
    // Continue counting even if the optional display is unavailable.
  } else {
    display.clearDisplay();
    display.setTextColor(SSD1306_WHITE);
  }

  // Restore the last saved count from ESP32 NVS.
  preferences.begin("visitor_data", false);
  occupantCount = preferences.getInt("count", 0);
  if (occupantCount < 0) occupantCount = 0;

  // Initialize debounce variables from the actual sensor levels.
  rawA = stableA = digitalRead(SENSOR_A_PIN);
  rawB = stableB = digitalRead(SENSOR_B_PIN);
  rawAChangedAt = rawBChangedAt = millis();

  updateOLEDDisplay();
  updateRelay();

  Serial.println("Visitor counter ready.");
}

// -------------------- Main loop --------------------------
void loop() {
  updateDebouncedInputs();
  handleDirectionalFSM();
  updateStatusLED();
  updateRelay();
```

```
}

// -------------------- Stable sensor sampling --------------
void updateDebouncedInputs() {
  const uint32_t now = millis();

  bool newRawA = digitalRead(SENSOR_A_PIN);
  bool newRawB = digitalRead(SENSOR_B_PIN);

  if (newRawA != rawA) {
    rawA = newRawA;
    rawAChangedAt = now;
  }
  if ((now - rawAChangedAt >= DEBOUNCE_MS) && stableA != rawA) {
    stableA = rawA;
  }

  if (newRawB != rawB) {
    rawB = newRawB;
    rawBChangedAt = now;
  }
  if ((now - rawBChangedAt >= DEBOUNCE_MS) && stableB != rawB) {
    stableB = rawB;
  }
}

// -------------------- Bi-directional FSM ------------------
void handleDirectionalFSM() {
  const uint32_t now = millis();

  // Sensors are active LOW.
  const bool sensorA = (stableA == LOW);
  const bool sensorB = (stableB == LOW);

  switch (currentState) {
    case IDLE:
      if (sensorA && !sensorB) {
        currentState = A_TRIGGERED;
        stateStartTime = now;
      } else if (sensorB && !sensorA) {
        currentState = B_TRIGGERED;
        stateStartTime = now;
      }
      break;

    case A_TRIGGERED:
      if (sensorB) {
        // A followed by B = entry.
        occupantCount++;
        saveOccupancyToFlash();
        updateOLEDDisplay();
        digitalWrite(STATUS_LED_PIN, HIGH);
        ledPulseActive = true;
        ledTurnOffAt = now + LED_PULSE_MS;
        currentState = IDLE;
      } else if (now - stateStartTime >= SEQUENCE_TIMEOUT_MS) {
        currentState = IDLE;
      }
      break;

    case B_TRIGGERED:
      if (sensorA) {
```

```
        // B followed by A = exit. Never allow a negative count.
        if (occupantCount > 0) {
          occupantCount--;
        }
        saveOccupancyToFlash();
        updateOLEDDisplay();
        digitalWrite(STATUS_LED_PIN, HIGH);
        ledPulseActive = true;
        ledTurnOffAt = now + LED_PULSE_MS;
        currentState = IDLE;
      } else if (now - stateStartTime >= SEQUENCE_TIMEOUT_MS) {
        currentState = IDLE;
      }
      break;
  }
}

// -------------------- Non-blocking LED pulse ---------------
void updateStatusLED() {
  if (ledPulseActive &&
      (int32_t)(millis() - ledTurnOffAt) >= 0) {
    digitalWrite(STATUS_LED_PIN, LOW);
    ledPulseActive = false;
  }
}

// -------------------- Relay control ------------------------
void updateRelay() {
  digitalWrite(RELAY_PIN, occupantCount > 0 ? HIGH : LOW);
}

// -------------------- NVS persistence ----------------------
void saveOccupancyToFlash() {
  preferences.putInt("count", occupantCount);
}

// -------------------- OLED display -------------------------
void updateOLEDDisplay() {
  // Display failure does not stop the counter.
  // Adafruit SSD1306 has no simple public "ready" flag, so begin()
  // is checked in setup and drawing calls are kept here.
  display.clearDisplay();
  display.setTextColor(SSD1306_WHITE);

  display.setTextSize(1);
  display.setCursor(0, 0);
  display.println("SMART VISITOR COUNTER");

  display.drawLine(0, 12, 127, 12, SSD1306_WHITE);

  display.setTextSize(1);
  display.setCursor(0, 22);
  display.print("Occupants:");

  display.setTextSize(3);
  display.setCursor(0, 34);
  display.print(occupantCount);

  display.setTextSize(1);
  display.setCursor(76, 39);
  display.print(occupantCount > 0 ? "ROOM ON" : "ROOM OFF");
```

```
  display.display();
}
```

6. Upload & Test Procedure

1. Connect ESP32 to the computer using a USB data cable.

2. Install the ESP32 board package and the two Adafruit libraries.

3. Select the ESP32 Dev Module board and the correct serial port in Arduino IDE.

4. Upload the sketch and open Serial Monitor at 115200 baud.

5. Connect the two IR sensors so their outputs are active LOW when a beam is interrupted.

6. Test entry: interrupt A first, then B within 1500 ms. Count should increase by one.

7. Test exit: interrupt B first, then A within 1500 ms. Count should decrease by one, but never below zero.

8. Reset or power-cycle the ESP32. The last saved occupancy count should be restored from NVS.

9. Verify relay output: HIGH when count is above zero; LOW when count is zero.

7. Important Engineering Notes

• **Relay polarity:** This firmware follows the uploaded document's active-HIGH relay definition. Confirm the actual relay board's input polarity before connecting a load.

• **Sensor arrangement:** Place Sensor A and Sensor B in sequence across the doorway. The direction logic assumes the person interrupts both beams in order.

• **Simultaneous triggering:** If both sensors become active before the FSM sees a first sensor, the current logic does not infer a direction; adjust physical spacing/alignment if this occurs.

• **Power-loss behavior:** The count is written to NVS after each confirmed sequence. NVS wear is generally suitable for event-based updates, but avoid saving continuously in the main loop.

• **OLED address:** The code uses I²C address 0x3C. Change SCREEN_ADDRESS if the display module uses a different address.

• **Mains safety:** 220–230 V AC can cause fatal electric shock or fire. Do not prototype mains wiring on a breadboard. Use a correctly rated, enclosed, fused relay arrangement and have mains-side work performed/checked by a qualified electrician. Keep mains and low-voltage PCB areas separated with appropriate creepage and clearance.

8. Scope & Limitations

This source implements the functions and GPIO assignments documented in the supplied project PDF. It does not establish that the pictured PCB has been electrically verified, that its traces meet mains clearance requirements, or that the relay/sensors are wired exactly as assumed. Bench-test with low-voltage loads first and verify the physical circuit before deployment.